DSR-1

WM-D6C Servo Replacement Module

Firmware Codebase Reference

Revision 0.1 | STM32G0B1KBU6 | Bare-Metal C | MIT License

1. Overview

The DSR-1 firmware replaces the failed capstan servo IC (Sony CX20084) in the WM-D6C professional cassette recorder with a digital PI servo loop running on an STM32G0B1KBU6 microcontroller. It also provides a USB CDC virtual COM port for live tuning and flash-backed persistent settings storage.

The firmware is written entirely in bare-metal C — no STM32Cube HAL, no RTOS, no dynamic memory allocation. Every peripheral register write is direct and documented. The only external dependency is the CMSIS device header (stm32g0b1xx.h), which provides named constants for register addresses.

1.1 File Inventory

File	Purpose
config.h	All tunable constants — edit here, rebuild
servo.c / servo.h	TIM2 input capture ISR, PI control law, DAC/PWM output, freeze/unfreeze
adc.c / adc.h	ADC DMA scan for RV601/RV602/RV603 pot wipers, S601 switch, adjusted target
flash.c / flash.h	Settings save/restore, CRC-32 validation, servo freeze coordination
usb_cdc.c / usb_cdc.h	USB CDC virtual COM port, enumeration, command parser, telemetry
main.c	Clock init, peripheral sequencing, boot status, main loop with gated WFI
Makefile	arm-none-eabi-gcc build, flash/DFU targets, CMSIS path discovery
STM32G0B1KBUx_FLASH.ld	Linker script — 126 KB firmware region, 2 KB settings sector reserved
startup_stm32g0b1xx.s	ST-provided CMSIS startup — stack, .data copy, .bss zero, calls main()

1.2 Memory Map

Address Range	Contents
0x08000000 – 0x0801EFFF	126 KB — firmware code and read-only data
0x0801F000 – 0x0801FFFF	2 KB — settings sector (reserved, never allocated to code)
0x20000000 – 0x20023FFF	144 KB RAM — stack, BSS, initialised data, USB PMA

1.3 Interrupt Priority Hierarchy

The priority assignment below is architectural and must not be changed without explicit justification. Cortex-M0+ uses lower numeric value = higher priority.

Priority	Handler	Rationale
0 (highest)	TIM2_IRQHandler	Servo ISR — must preempt everything including USB
1	USB_UCPD1_2_IRQHandler	USB enumeration and CDC data transfers
2 (lowest)	SysTick (currently unused)	Available for future use

1.4 Boot Sequence

Total time from power-on reset to servo loop running is approximately 2 ms — well inside the ~300 ms mechanical spin-up time of the WM-D6C capstan flywheel.

Time	Event
t = 0	Power-on reset released, execution begins at Reset_Handler in startup file
t ≈ 0.3 ms	clock_init(): HSI16 → PLL → 64 MHz system clock
t ≈ 0.3 ms	flash_load(): settings loaded from flash (or defaults used)
t ≈ 0.5 ms	adc_init(): DMA continuous scan of RV601/RV602/RV603 starts
t ≈ 0.5 ms	servo_init(): TIM2, DAC pre-loaded to DAC_CENTER, NVIC priority 0
t ≈ 0.5 ms	usb_cdc_init(): HSI48, CRS, USB peripheral, DP pull-up asserted
t ≈ 0.5 ms	__enable_irq(): global interrupts unmasked
t ≈ 2 ms	First FG edge arrives, TIM2_IRQHandler fires, servo loop running
t ≈ 300 ms	Motor reaches operating speed, servo locked to target period

2. config.h — Tunable Constants

config.h is the single file that should be edited for normal configuration. It is divided into numbered sections. Rebuild after any change — all source files include this header.

2.1 Hardware Variant Selection

Define SERVO_OPTION_B to select the PWM + NPN level-shift output stage for machines where the Q601 base voltage exceeds 3.3 V. Leave undefined (default) for direct DAC drive.

```
/* #define SERVO_OPTION_B */   /* Uncomment for Option B output stage */
```

WARNING: The output stage must be selected based on bench measurement of the Q601 base voltage range on the specific unit. See signal-chain-analysis.md §2.

2.2 FG Target Period

TARGET_PERIOD_DEFAULT is the most critical constant in the firmware. It must be measured on the bench using test tape WS-48B — the placeholder value of 25,600 ticks (2500 Hz) will not be correct for every unit.

```
#define FG_TARGET_HZ_NOMINAL    2500UL
#define TARGET_PERIOD_DEFAULT   (SYSCLK_HZ / FG_TARGET_HZ_NOMINAL)  /* 25,600 ticks */

/* Bench procedure:
 * 1. Insert test tape WS-48B, play mode
 * 2. Confirm 3 kHz on LINE OUT (= correct speed)
 * 3. Measure FG901 pulse frequency at that speed
 * 4. TARGET_PERIOD_DEFAULT = 64,000,000 / measured_Hz */
```

2.3 PI Gain Constants

Gain constants are stored in Q16 fixed-point format (value × 65536). Starting values are engineering estimates — bench tuning with the USB CDC interface is required for final calibration.

Constant	Value and Meaning
KP_Q16_DEFAULT	9830 (Kp = 0.15 — moderate proportional response)
KI_Q16_DEFAULT	524 (Ki = 0.008 — slow integral, ~50 ms time constant)
KP_Q16_STEP	655 (USB CDC p+/p- step ≈ ±0.01 float)
KI_Q16_STEP	66 (USB CDC i+/i- step ≈ ±0.001 float)
KP_Q16_MAX	65536 (Kp = 1.0 — hard upper limit)
KI_Q16_MAX	6554 (Ki = 0.1 — hard upper limit)

2.4 Anti-Windup and DAC Limits

Constant	Value and Meaning
INTEGRAL_LIMIT	1,280,000 (TARGET_PERIOD × 50 — bounds integral during spin-up)
DAC_CENTER	2048 (12-bit midscale, ~1.65 V, normal operating point)
DAC_MIN	100 (near-maximum motor drive — 0.08 V on PA4)
DAC_MAX	3995 (near-minimum motor drive — 3.22 V on PA4)
MIN_PERIOD_TICKS	6400 (rejects FG periods shorter than 10,000 Hz equivalent)

2.5 Compile-Time Assertions

config.h includes three _Static_assert checks that catch configuration errors at compile time rather than runtime:

- **KI_Q16_DEFAULT × INTEGRAL_LIMIT < 2^{31}** — prevents overflow in the integral multiplication inside the ISR
- **DAC_MIN < DAC_CENTER < DAC_MAX** — ensures the output clamp range is sane
- **TARGET_PERIOD_DEFAULT > MIN_PERIOD_TICKS** — ensures the noise rejection threshold is below the normal operating period

3. servo.c / servo.h — Servo Loop

3.1 Peripherals Owned

Peripheral	Role
TIM2	Free-running 32-bit counter at 64 MHz, CH1 input capture on PA0 (FG901 signal)
DAC1	Channel 1 on PA4 — 12-bit analog output to Q601 base (Option A)
TIM3	Channel 1 on PA6 — PWM output for Option B NPN level-shift stage (compile-time only)

3.2 Module-Private State

Two static volatile variables are shared between TIM2_IRQHandler and the freeze/unfreeze functions. They are file-scope (not local to the ISR) so that servo_freeze() and servo_unfreeze() can access them directly without indirection.

```
static volatile uint32_t last_capture = 0;  /* Counter value at last FG rising edge */
static volatile int32_t  integral     = 0;  /* PI integral accumulator              */
```

3.3 Public Globals

These volatile variables are written by servo.c and read by usb_cdc.c for telemetry, or written by usb_cdc.c and read by the ISR for live tuning. All are 32-bit aligned — reads and writes are atomic on Cortex-M0+ without a mutex.

Variable	Description
g_kp_q16	Proportional gain in Q16 format. Written by USB CDC p+/p-, read by ISR.
g_ki_q16	Integral gain in Q16 format. Written by USB CDC i+/i-, read by ISR.
g_target_period	FG target period in ticks. Written by USB CDC f+/f-, read by ISR.
g_telemetry	ServoTelemetry struct. Written by ISR, read by USB CDC for telemetry output.

3.4 servo_init()

Called once from main() after clock_init(). Configures all servo-related peripherals. The TIM2 ISR does not fire until __enable_irq() is called later in main().

Key configuration decisions:

- **TIM2->PSC = 0:** No prescaler — every tick is 15.625 ns at 64 MHz, giving maximum period resolution
- **TIM2->ARR = 0xFFFFFFFF:** Full 32-bit free-running counter — wraps approximately every 67 seconds, handled transparently by unsigned subtraction in the ISR
- **DAC1->DHR12R1 = DAC_CENTER:** Pre-loaded before NVIC is enabled — the motor sees a safe mid-scale drive from the first moment the DAC is active
- **NVIC priority 0:** Highest possible — the servo ISR must preempt everything including USB

3.5 TIM2_IRQHandler() — The Servo ISR

This is the heart of the firmware. It runs on every rising edge of the FG901 optical sensor signal. At correct tape speed the FG rate is approximately 2500 Hz, so this ISR executes every ~400 µs.

Execution budget: < 30 clock cycles, approximately 470 ns. The ISR must complete before the next FG edge to avoid missing pulses.

3.5.1 Period Measurement

```
uint32_t now    = TIM2->CCR1;       /* Hardware-captured counter value at FG edge */
uint32_t period = now - last_capture; /* Unsigned subtraction handles 32-bit wrap   */
last_capture    = now;
```

The hardware stores the TIM2 counter value at the exact moment of the FG rising edge in CCR1 — no software latency involved. Unsigned 32-bit subtraction handles counter wrap-around correctly without special-casing.

3.5.2 Noise Rejection

```
if (period < MIN_PERIOD_TICKS) {
    TIM2->SR &= ~TIM_SR_CC1IF;
    return;
}
```

Implausibly short periods (faster than 10,000 Hz FG equivalent) are discarded. This prevents noise on the FG line at power-on, or contact bounce in the optical sensor, from corrupting the integral accumulator during the period before the capstan reaches speed.

3.5.3 PI Control Law

```
/* Error: positive = motor too slow (period too long) */
int32_t error = (int32_t)period - (int32_t)target;

/* Integral with anti-windup clamp */
integral += error;
if (integral >  INTEGRAL_LIMIT) integral =  INTEGRAL_LIMIT;
if (integral < -INTEGRAL_LIMIT) integral = -INTEGRAL_LIMIT;

/* Q16 fixed-point output — no floating point */
int32_t output = DAC_CENTER
               - ((kp * error)    >> 16)
               - ((ki * integral) >> 16);
```

The PNP transistor Q601 convention drives the sign: a positive error (motor too slow) subtracts from DAC_CENTER, reducing the base voltage, increasing collector current, and speeding the motor up. This is the opposite sign convention from an NPN drive stage.

Q16 arithmetic: multiplying a Q16-scaled gain constant by a plain integer (Q0) gives a Q16 result. Right-shifting 16 positions recovers the integer result. This is equivalent to dividing by 65536 but executes as a single-cycle barrel shift on Cortex-M0+ rather than a multi-cycle division. All arithmetic fits within int32_t — see config.h §2.5 for overflow analysis.

3.6 servo_freeze() and servo_unfreeze()

Flash erase operations on the single-bank STM32G0B1 stall instruction fetch for ~2 ms. Without mitigation, the ISR would miss 4–5 FG edges, the integral would accumulate stale error, and the motor would receive a large correction spike on resumption.

The freeze/unfreeze pair eliminates this problem:

Function	Action
servo_freeze()	Disables TIM2 IRQ; clears integral; DAC output unchanged at last good value
servo_unfreeze()	Seeds last_capture = TIM2->CNT; re-enables TIM2 IRQ

Seeding last_capture with TIM2->CNT is critical. Without it, the first post-freeze ISR would compute period = CCR1 - 0 (or the stale pre-freeze value), producing an enormous error that would saturate the output stage. With the seed, the first delta is a normal sub-400 µs FG period and the loop re-locks within 1–2 FG cycles.

4. adc.c / adc.h — Speed Trim ADC

4.1 Peripherals Owned

Peripheral	Role
ADC1	3-channel scan: PA1 (IN1), PA2 (IN2), PA3 (IN3)
DMA channel 1	Circular transfer from ADC1 DR → g_adc_raw[3] — autonomous, no CPU involvement
PA7 GPIO	S601 Speed Tune enable switch — input with pull-down

4.2 ADC Channel Mapping

Index / Symbol	Pin	Function
g_adc_raw[0] ADC_IDX_RV601	PA1 / ADC_IN1	Base speed calibration trim (factory-set, covered plug)
g_adc_raw[1] ADC_IDX_RV602	PA2 / ADC_IN2	User Speed Tune slider — front panel control
g_adc_raw[2] ADC_IDX_RV603	PA3 / ADC_IN3	Speed Tune range scalar — sets RV602 sensitivity

4.3 Configuration Choices

- **Sampling time: 239.5 cycles** — required because pot source impedance can reach ~24 kΩ at mid-travel. At 16 MHz ADC clock this is 15 µs per channel, 45 µs for a complete 3-channel scan.
- **16× oversampling with right-shift 4** — improves effective resolution from 12-bit to ~14-bit. Effective update rate per channel ≈ 1300 Hz, well above any hand motion on the speed trim pots.
- **DMA circular mode** — ADC runs autonomously. g_adc_raw[] is always current. The servo ISR reads it without waiting or synchronising.

4.4 adc_get_adjusted_target()

Called from TIM2_IRQHandler on every FG edge. Returns the servo target period adjusted by all three pot readings and the S601 switch state.

```
/* RV601: base trim — signed offset centred at ADC midscale (2048) */
int32_t rv601_trim   = ((rv601 - 2048) * BASE_TRIM_SCALE) / 2048;

/* RV603: range scalar — unsigned, 0 to SPEED_RANGE_MAX */
int32_t rv603_range  = (rv603 * SPEED_RANGE_MAX) / 4095;

/* RV602: Speed Tune — applied only if S601 switch is HIGH */
int32_t rv602_offset = 0;
if (GPIOA->IDR & (1U << 7)) {
    rv602_offset = ((rv602 - 2048) * rv603_range) / 2048;
}

/* Combine and clamp to safe range */
int32_t adjusted = (int32_t)g_target_period + rv601_trim + rv602_offset;
```

BENCH MEASUREMENT REQUIRED: Pot wiper voltages must be measured on the specific unit before connecting to the STM32. If any wiper can exceed 3.3 V, voltage dividers must be added. Exceeding VDD + 0.3 V permanently damages the ADC input.

5. flash.c / flash.h — Settings Storage

5.1 Settings Block Layout

The settings block is 32 bytes, 8-byte aligned (required for STM32G0B1 double-word flash writes). It occupies the first 32 bytes of the last 2 KB page at 0x0801F000.

Offset	Size	Field
0x00	4 bytes	magic — SETTINGS_MAGIC sentinel (0x44535231, ASCII "DSR1")
0x04	4 bytes	kp_q16 — Proportional gain in Q16 format
0x08	4 bytes	ki_q16 — Integral gain in Q16 format
0x0C	4 bytes	target_period — FG target in 64 MHz ticks
0x10	4 bytes	crc32 — CRC-32 of bytes 0x00–0x0F (ISO 3309 polynomial)
0x14	12 bytes	reserved — written as 0xFFFFFFFF (maintains double-word alignment)

5.2 Validation on Load

flash_load() applies three independent checks before accepting stored values:

- Magic value must equal SETTINGS_MAGIC (0x44535231)
- CRC-32 of the first 16 bytes must match the stored crc32 field
- Kp, Ki, and target_period must be within the safe ranges defined in config.h

The third check catches the pathological case where a partially-erased page passes CRC by chance — a corrupt magic and a few flipped bits are astronomically unlikely to produce a valid CRC, but range validation adds a second independent layer at zero cost.

5.3 Flash Write Procedure

The STM32G0B1 flash controller requires double-word (64-bit) writes — two consecutive 32-bit words must be written before the programming latch fires. Writing a single word and stopping leaves the cell in a partially programmed state.

```
static void flash_write_dword(uint32_t addr, uint32_t lo, uint32_t hi)
{
    FLASH->CR |= FLASH_CR_PG;
    *(__IO uint32_t *)(addr)     = lo;   /* Write lower word first */
    *(__IO uint32_t *)(addr + 4) = hi;   /* Upper word triggers latch */
    flash_wait_busy();
    FLASH->CR &= ~FLASH_CR_PG;
}
```

5.4 Servo Freeze During Save

flash_save() calls servo_freeze() before flash_unlock() and servo_unfreeze() after flash_lock(). The motor runs at constant open-loop drive for the ~2 ms erase window. The sequence is:

```
servo_freeze();        /* Disable ISR, latch DAC, clear integral         */
flash_unlock();        /* Write KEYR sequence                             */
flash_erase_page(62);  /* ~2 ms stall — motor at constant drive          */
/* write 4 double-words */
flash_lock();          /* Set LOCK bit                                    */
servo_unfreeze();      /* Seed last_capture, re-enable ISR               */
/* verify readback */
```

5.5 The r / s Command Separation

flash_reset_defaults() (triggered by the r USB CDC command) only writes to SRAM. It does not touch flash. The user must follow with s to persist. This prevents an accidental r command during calibration from permanently overwriting tuned constants — there is no undo for a flash erase.

6. usb_cdc.c / usb_cdc.h — USB CDC Interface

6.1 USB Hardware Configuration

The STM32G0B1 implements crystal-less USB using an internal 48 MHz RC oscillator (HSI48) trimmed by the Clock Recovery System (CRS). The CRS measures HSI48 tick counts between USB SOF packets (sent by the host every 1.000 ms ±250 ppm) and automatically adjusts the HSI48 trim register to maintain accuracy within USB specification.

The system clock (64 MHz from HSI16/PLL) and the USB clock (48 MHz from HSI48/CRS) are completely independent. CRS trimming adjustments cannot perturb the servo loop timing.

6.2 PMA (Packet Memory Area) Layout

The STM32 USB peripheral uses a dedicated 2 KB SRAM (PMA) for endpoint buffers, accessed through a 16-bit bus at 0x40009800. The DSR-1 uses 256 bytes:

PMA Offset	Contents
0x000 – 0x03F	EP0 TX (64 bytes) — control endpoint, descriptor responses
0x040 – 0x07F	EP0 RX (64 bytes) — control endpoint, SETUP packets
0x080 – 0x0BF	EP1 TX (64 bytes) — CDC bulk IN, telemetry to host
0x0C0 – 0x0FF	EP2 RX (64 bytes) — CDC bulk OUT, commands from host

6.3 USB Descriptors

The device presents as a standard USB CDC ACM device, VID 0x0483 / PID 0x5740 (ST demo identifiers). No driver installation is required on Windows 10+, macOS, or Linux. The configuration descriptor declares two interfaces: a CDC Control interface (with a non-functional interrupt IN endpoint required by the spec) and a CDC Data interface with bulk IN (EP1) and bulk OUT (EP2).

6.4 Receive Path

Bytes received from the host on EP2 are copied from PMA into a 256-byte ring buffer (rx_buf[]) by the USB ISR. The main loop drains rx_buf[] via usb_cdc_poll() and dispatches each byte to the command parser. The ring buffer decouples ISR and main loop — the ISR never blocks and the main loop processes at its own rate.

6.5 Transmit Path

All transmit operations copy data into PMA at EP1_TX_ADDR and set EP1 status to VALID. The tx_busy flag is set while a transfer is in progress and cleared by the EP1 IN completion interrupt. Callers check tx_busy before transmitting — if busy, the transmission is silently dropped. This is acceptable for telemetry (the next update will be sent shortly) but means the banner and save-result messages must be sent when tx_busy is clear.

6.6 Command Interface

Command	Effect
p+ / p-	Adjust Kp by ±KP_Q16_STEP (≈ ±0.01 float). Takes effect on next ISR execution.
i+ / i-	Adjust Ki by ±KI_Q16_STEP (≈ ±0.001 float). Takes effect on next ISR execution.
f+ / f-	Adjust target period by ±1 tick. Fine-tune master speed reference.
t	Send one telemetry line immediately.
T	Toggle continuous telemetry mode. Any other key stops it.
s	Save current Kp, Ki, target_period to flash (calls flash_save()).
r	Restore compiled-in defaults to SRAM only — does not write flash.
?	Print command reference.

6.7 Telemetry Format

The t command (and continuous T mode) sends a single line per update:

```
FG:25612 Hz:2499 Tgt:25600 Err:+12 Int:-847 DAC:2034 Kp:9830(0.150) Ki:524(0.008) RV1:2048
RV2:1923 RV3:2100
```

The telemetry formatter uses no printf, no sprintf, and no floating-point — integer-only arithmetic with custom decimal formatters. The Q16 values are displayed as both their raw integer and float-equivalent forms (e.g. 9830(0.150)).

6.8 Boot Status Banner

When the host connects and SET_CONFIGURATION completes, banner_pending is set. The next usb_cdc_poll() call sends either a clean connection message or a prominent warning if flash_load() returned FLASH_ERR_INVALID:

```
*** WARNING: Flash settings missing or corrupt — running defaults ***
*** Calibrate and type 's' to save before use                     ***
```

This ensures that a unit running on defaults (first boot, or after flash corruption) cannot be used for calibration-sensitive recording without the operator being explicitly notified.

7. main.c — Startup and Main Loop

7.1 clock_init()

Transitions the system clock from the power-on default (HSI16, 16 MHz) to the target (64 MHz via PLL). The sequence must follow a strict order — flash latency must be set before the clock switch to avoid a read error at the higher frequency.

```
/* Step 1: HSI16 on and stable (default at reset, explicit for safety) */
RCC->CR |= RCC_CR_HSION;
while (!(RCC->CR & RCC_CR_HSIRDY)) {}

/* Step 2: Configure PLL: HSI16 × 8 / 2 = 64 MHz */
RCC->PLLCFGR = RCC_PLLCFGR_PLLSRC_HSI | (0 << PLLM) | (8 << PLLN) | (1 << PLLR) | PLLREN;

/* Step 3: Enable PLL */
RCC->CR |= RCC_CR_PLLON;
while (!(RCC->CR & RCC_CR_PLLRDY)) {}

/* Step 4: Flash latency BEFORE clock switch — 2 wait states at 64 MHz */
FLASH->ACR = (2 << LATENCY) | PRFTEN | ICEN;
while ((FLASH->ACR & LATENCY) != (2 << LATENCY)) {}  /* Verify accepted */

/* Step 5: Switch to PLLR */
RCC->CFGR = (RCC->CFGR & ~SW) | SW_PLLRCLK;
while ((RCC->CFGR & SWS) != SWS_PLLRCLK) {}
```

7.2 Peripheral Initialisation Order

The order matters — each step assumes the previous is complete. No interrupt can fire before __enable_irq() because the Cortex-M0+ boots with interrupts masked.

Step	Rationale
1. clock_init()	Must be first — all peripherals derive their clock from SYSCLK
2. flash_load()	Must be before servo_init() — loads g_kp_q16, g_ki_q16, g_target_period
3. adc_init()	Before servo_init() — DMA scan starts; first conversion complete by first FG edge
4. servo_init()	After ADC — TIM2 and DAC configured; NVIC registered at priority 0
5. usb_cdc_init()	After servo — USB/CRS/HSI48 configured; DP pull-up asserted
6. usb_cdc_set_boot_status()	After usb_cdc_init(), before __enable_irq() — stores flash result for banner
7. __enable_irq()	Last — all peripherals ready before any interrupt fires

7.3 Main Loop with Gated WFI

```
for (;;) {
    usb_cdc_poll();

    /* Only sleep if there is genuinely nothing pending */
    if (usb_cdc_idle()) {
```

```
            __WFI();
        }
    }
```

WFI is gated by usb_cdc_idle() which returns 0 (do not sleep) if any of these conditions hold:

- rx_buf[] contains unprocessed bytes
- banner_pending is set (host just connected)
- telem_continuous is active and TX is free

Without this gate, a USB interrupt could wake the CPU, queue data into rx_buf[], return, and the CPU would immediately re-enter WFI before usb_cdc_poll() had a chance to drain the buffer. The gated pattern ensures the main loop always services any pending work before sleeping.

8. Makefile

8.1 Toolchain Requirements

Tool	Purpose
arm-none-eabi-gcc	GCC cross-compiler for ARM bare-metal (version ≥ 10.0)
arm-none-eabi-objcopy	Converts ELF to .bin and .hex
arm-none-eabi-size	Reports section sizes
openocd	SWD flash programming (make flash) — optional
dfu-util	USB DFU flash programming (make dfu) — optional

Installation:

- macOS: brew install --cask gcc-arm-embedded
- Ubuntu: sudo apt install gcc-arm-none-eabi
- Windows: Download from developer.arm.com/downloads/-/gnu-rm

8.2 CMSIS Header Discovery

The Makefile searches three standard locations for the STM32G0 CMSIS headers before failing. Override by setting CMSIS_DIR on the command line:

```
make CMSIS_DIR=/path/to/STM32Cube_FW_G0_V1.6.1/Drivers/CMSIS
```

If headers are not found in standard locations, copy them to firmware/cmsis/ and the Makefile will find them automatically.

8.3 Build Targets

Target	Action
make	Build firmware.elf, firmware.bin, firmware.hex, print size report
make flash	Flash via SWD using OpenOCD with ST-Link interface
make dfu	Flash via USB DFU (device must be in DFU mode — hold TP1/BOOT0)
make clean	Remove all build artefacts from build/obj/ and build/
make size	Print detailed section size breakdown
make disasm	Generate annotated disassembly listing (firmware.lss)
make print-VARNAME	Print the value of any Makefile variable for debugging

8.4 Compiler Flags

Flag	Purpose
-O2	Optimisation level — as specified in module datasheet §6.4
-ffunction-sections	Places each function in its own section for dead-code elimination

Flag	Purpose
-fdata-sections	Places each variable in its own section
-Wl,--gc-sections	Linker removes unreferenced sections — keeps binary small
-nostdlib	No implicit libc linkage
--specs=nano.specs	Link newlib-nano — provides only memcpy/memset, no printf heap
-g3 -gdwarf-4	Full debug info in ELF (stripped from .bin by objcopy)

9. STM32G0B1KBUx_FLASH.ld — Linker Script

9.1 Memory Regions

Region	Definition
FLASH (rx)	0x08000000, LENGTH = 126K — firmware code only; settings sector excluded
RAM (xrw)	0x20000000, LENGTH = 144K — full RAM available to firmware

The settings sector at 0x0801F000 is deliberately absent from the FLASH region. The linker has no visibility of that address range and can never place code or read-only data there. The 2 KB page is reserved solely for flash.c.

9.2 Section Order in Flash

Section	Notes
.isr_vector	Must be first at exactly 0x08000000 — Cortex-M0+ reads stack pointer and reset vector from offsets 0 and 4
.text	All code and read-only data (-ffunction-sections allows --gc-sections to remove unused functions)
.ARM.extab/.ARM	Exception unwinding tables — required for correct C++ compatibility even though C++ is not used
.init_array	GCC init hooks — present for toolchain compatibility, not used by this firmware
.data (LMA)	Initialised variable images stored in flash, copied to RAM by startup.s

9.3 Section Order in RAM

Section	Notes
.data (VMA)	Initialised variables — copied from flash by startup.s before main()
.bss	Uninitialised variables — zeroed by startup.s before main()
Stack	Grows downward from _estack = 0x20024000 (top of 144 KB RAM)

9.4 Settings Sector Guard

The linker script includes an ASSERT that fails the build if the firmware image overflows into the settings sector:

```
ASSERT(. <= 0x0801F000,
    "ERROR: Firmware image exceeds 126 KB and overlaps the settings sector.")
```

At the current firmware size of ~28 KB against a 126 KB budget, there is approximately 98 KB of headroom. The assert exists for the long term — a future contributor adding a large feature will get a clear build failure rather than silent settings corruption.

9.5 Stack Size Verification

```
_Min_Stack_Size = 0x800;   /* 2 KB — verified at link time */
_Min_Heap_Size  = 0x000;   /* No heap — all memory statically allocated */
```

If stack + heap + BSS + data does not fit within 144 KB RAM, the linker emits "region RAM overflowed" and the build fails. With ~2 KB RAM usage against 144 KB available, this check exists only as future-proofing.

10. Concurrency and Shared State

The firmware has two execution contexts: the TIM2 ISR and the main loop. There is no RTOS, no mutex, no semaphore. Correct sharing of state between the two contexts relies on three properties of the Cortex-M0+ architecture:

10.1 Atomic 32-bit Reads and Writes

On Cortex-M0+, a 32-bit load or store to a naturally-aligned address is a single instruction and therefore atomic — it cannot be interrupted mid-execution. All shared variables (g_kp_q16, g_ki_q16, g_target_period, g_telemetry fields, last_capture, integral) are 32-bit aligned. This means:

- The main loop can read g_telemetry fields without disabling interrupts — it will always see a complete 32-bit value, never a torn read
- usb_cdc.c can write g_kp_q16 without disabling interrupts — the ISR will see either the old or new value, never a mix
- The ISR writes g_telemetry fields — the main loop will see either the old complete snapshot or the new complete snapshot

10.2 Interrupt Priority and Preemption

On Cortex-M0+, a lower-priority interrupt (USB, priority 1) cannot preempt a higher-priority interrupt (TIM2, priority 0). The servo ISR is never suspended by USB activity. Conversely, the USB ISR can be preempted by the servo ISR if a FG edge arrives mid-USB-processing — this is correct and harmless since they do not share state.

10.3 Cases Requiring Explicit Disable

Three operations require explicitly disabling the TIM2 interrupt because they involve multi-step modifications to shared state:

Function	Reason for Explicit Disable
servo_reset_integral()	Zeroes integral — disable TIM2 to prevent ISR from reading a half-zeroed value during a hypothetical future multi-word integral (currently int32_t, so atomic, but future-safe)
servo_freeze()	Disables TIM2 IRQ, then clears integral — the disable itself is the critical operation
servo_unfreeze()	Writes last_capture, then re-enables TIM2 IRQ — the write must complete before any ISR can execute

10.4 The Volatile Keyword

All variables written by the ISR and read by the main loop (or written by the main loop and read by the ISR) are declared volatile. This prevents the compiler from caching values in registers across the ISR boundary. Without volatile, -O2 optimisation would be free to read g_kp_q16 once at loop entry and never re-read it — making live tuning via USB CDC ineffective.

11. Bench Calibration Procedure

This section describes the procedure for initial calibration of a DSR-1 module after installation. It assumes the module is installed, wired, and the USB-C cable is connected to a computer running a serial terminal.

11.1 Prerequisite Bench Measurements

These measurements must be taken before power-on with the STM32 connected:

Measurement	Purpose
Q601 base voltage range	Determines Option A vs Option B output stage. Measure with original CX20084 installed (or estimated from service manual). If 0–3.3 V: Option A. If > 3.3 V: Option B.
FG901 signal swing	Determines R3/R4 divider values. Measure at TP_FG on the original board during playback. Must be conditioned to < 3.3 V at PA0.
CP304 output (B+3)	Measure B+3 rail on battery power. Should be ~10.8 V. If absent or incorrect, CP304 must be replaced with MT3608 before module installation.
Pot wiper voltages	Measure RV601, RV602, RV603 wiper outputs at full travel. If any exceeds 3.3 V, voltage dividers must be added before ADC connection.
IC601 pin 7 net	Confirm ~4.4 V during playback. This confirms the 10 kΩ + 22 kΩ divider sizing to PA5.

11.2 TARGET_PERIOD Calibration

- Insert test tape WS-48B, select Play mode
- Connect oscilloscope to LINE OUT
- Adjust using f+/f- commands until 3 kHz tone is confirmed on LINE OUT
- Note the FG period displayed in telemetry (FG: field)
- Update TARGET_PERIOD_DEFAULT in config.h with this value, rebuild, and reflash
- Send s to save the calibrated value to flash

NOTE: The f+/f- commands adjust target_period one tick at a time (15.6 ns at 64 MHz). One tick corresponds to approximately 0.004% speed change at 2500 Hz. Fine calibration requires patience — use continuous telemetry mode (T) while adjusting.

11.3 PI Gain Tuning

The default values (Kp = 0.15, Ki = 0.008) are conservative starting points. Tuning procedure:

- Start with defaults. Play test tape, enable continuous telemetry (T).
- Observe the Err: and Int: fields. A well-tuned servo shows Err near zero with slow, smooth integral drift.
- If speed oscillates audibly: reduce Kp with p- commands. Oscillation indicates excessive proportional gain.
- If motor is sluggish to lock after pressing Play: increase Kp with p+ commands.

- If there is a sustained speed offset that is slow to correct: increase Ki with i+ commands.
- If the servo hunts slowly around the target: reduce Ki with i- commands.
- Once stable: send s to save constants to flash.

11.4 Verifying the Flash Save

After sending s, the terminal will display either "Saved" or "Save ERR". On next power-on, the connection banner will show "Settings: loaded from flash" confirming the saved values are active. If the banner shows the corruption warning, repeat calibration and save.

12. Contributor Guidelines

12.1 Non-Negotiable Rules

These rules are architectural requirements enforced at review. They are not preferences.

- **No floating-point in TIM2_IRQHandler.** The Cortex-M0+ has no FPU. Software float takes 20–40 cycles per operation; the ISR budget is 30 cycles total.
- **No HAL, no LL, no RTOS.** Register-level C only. Any proposed abstraction that adds flash consumption or execution overhead requires explicit justification.
- **TIM2 interrupt priority 0 is non-negotiable.** Raising USB priority above TIM2 allows USB processing to delay servo execution.
- **Double-word writes only in flash.c.** Single-word writes to flash corrupt the cell on STM32G0B1.
- **CHANGELOG.md is mandatory.** Every resistor value change, constant update, and layout revision must be described in terms of what was observed and why the change was made.

12.2 Adding a New Feature

The recommended approach for any new feature:

- Add constants to config.h with comments explaining their derivation
- Implement in a new .c/.h pair — keep servo.c, adc.c, flash.c, and usb_cdc.c focused on their current responsibilities
- If the feature needs to share data with the servo ISR, follow the volatile + 32-bit-aligned pattern in §10
- If the feature needs to inhibit the servo ISR, use servo_freeze() / servo_unfreeze()
- Add a build-time _Static_assert for any constant that must satisfy a numerical constraint
- Update CHANGELOG.md before submitting

12.3 Porting to a Different Machine

The variant system is designed to make machine-specific ports a bounded task. A complete port requires three artefacts in docs/variants/:

- A schematic diff from the DSR-1 reference showing any signal conditioning changes
- A harness pinout diagram mapping the 8-pin J4 connector to the target machine
- The calibrated FG_TARGET_HZ value for that machine's capstan speed

Core hardware (STM32, power supply, USB) and firmware (servo algorithm, USB CDC, flash storage) are unchanged by a variant port. Only config.h constants and any machine-specific signal conditioning differ.

12.4 Porting Firmware to an MCU with FPU

If porting to a Cortex-M4F or M7 with a hardware FPU, the Q16 arithmetic in TIM2_IRQHandler can be replaced with float:

```
/* Q16 version (Cortex-M0+, no FPU) */
int32_t output = DAC_CENTER
               - ((kp * error)    >> 16)
               - ((ki * integral) >> 16);

/* Float version (Cortex-M4F or higher, hardware FPU) */
float output = DAC_CENTER
             - (kp_float * error)
             - (ki_float * integral);
```

The Q16 version is retained in DSR-1 because it is the correct choice for Cortex-M0+ and because the explicit fixed-point arithmetic makes the gain scaling and precision visible in a way that float can obscure. Do not change it without changing the target MCU.